QAOA on simple instance of maxcut

August 23, 2021

The purpose of this notebook is to give you a rough understanding of the Quantum Approximate Optimization Algorithm [https://arxiv.org/abs/1411.4028]. To make it as simple as possible, we will use the simplest (and roughest) version of this algorithm, and we will apply it to a simple instance of the maximum cut problem [https://en.wikipedia.org/wiki/Maximum_cut].

In the first two blocks, we define helpful variables and functions to make the later code more readable:

[1]:
```
import numpy as np

plus = np.array([1,1])/np.sqrt(2)
X = np.array([[0,1],[1,0]])
Z = np.array([[1,0],[0,-1]])
I = np.identity((2))
```

[2]:
```
def plus_kron_k(k):
    s = np.kron(plus, plus)
    for _ in range(k-2):
        s = np.kron(s, plus)
    return s

def zz(n,j,k):
    chain = 'I'*(n-2)
    chain = chain[:j-1]+'Z' + chain[j-1:k-2] + 'Z' + chain[k-2:]
    ZZ = 1
    for op in chain:
        ZZ = np.kron(ZZ,I) if op=='I' else np.kron(ZZ,Z)
    return ZZ

def x(n,k):
    op = 1
    for i in range(n):
        op = np.kron(op,I) if i!=(k-1) else np.kron(op,X)
    return op

def i(n):
    return np.identity(2**n)
```

```
def exp_igZZ(n,j,k,g):
    return np.cos(g)*i(n) - 1j*np.sin(g)*zz(n,j,k)

def exp_ibX(n,k,b):
    return np.cos(b)*i(n) + 1j*np.sin(b)*x(n,k)


def exp_igC(n,edges,g):
    res = np.identity(2**6)
    for edge in edges:
        res = exp_igZZ(n, edge[0], edge[1], g) @ res
    return res

def exp_ibB(n,b):
    res = np.identity(2**6)
    for q in range(n):
        res = exp_ibX(n,q,b) @ res
    return res
```

1 Simple instance of MaxCut

[3]:
```
# GRAPH
n = 6
edges = [[1,3],[1,5],[1,6],[2,4],[2,5],[2,6],[3,4],[3,5],[5,6]]
```

[4]:
```
# cut is the number of disagreeing edges (0-1)
def cut(label):
    res = 0
    for edge in edges:
        if label[edge[0]-1] != label[edge[1]-1]:
            res += 1
    return res
```

[5]:
```
# brute force -> try all possibilities, choose the best
best = [0,'000000']
for label in [f'{i:06b}' for i in range(2**6)]:
    mc = cut(label)
    if mc > best[0]:
        best = [mc, label]
best
```

[5]: [7, '000111']

2 Initial state $s = |+\rangle^{\otimes k} = \frac{1}{\sqrt{2^n}} \sum_{z \in \{0,1\}^n} |z\rangle$:

[6]:
```
s = plus_kron_k(n)
#observe that |s> is the uniform superposition over all computational basis states
s
```

[6]:
```
array([0.125, 0.125, 0.125, 0.125, 0.125, 0.125, 0.125, 0.125, 0.125,
       0.125, 0.125, 0.125, 0.125, 0.125, 0.125, 0.125, 0.125, 0.125,
       0.125, 0.125, 0.125, 0.125, 0.125, 0.125, 0.125, 0.125, 0.125,
       0.125, 0.125, 0.125, 0.125, 0.125, 0.125, 0.125, 0.125, 0.125,
       0.125, 0.125, 0.125, 0.125, 0.125, 0.125, 0.125, 0.125, 0.125,
       0.125, 0.125, 0.125, 0.125, 0.125, 0.125, 0.125, 0.125, 0.125,
       0.125, 0.125, 0.125, 0.125, 0.125, 0.125, 0.125, 0.125, 0.125,
       0.125])
```

3 p=1 QAOA(g, b)

[7]:
```
# This function will simulate p=1 QAOA <how_many_repetitions> times.
# It will return <how_many_repetitions> strings representing the measurement results.
def p1_qaoa(n,edges,g,b,how_many_repetitions):
    # initial state
    s = plus_kron_k(n)
    # evolve with C for time g
    s = exp_igC(n,edges,g) @ s
    # evolve with B for time b
    s = exp_ibB(n,b) @ s
    # at last, we project s to the computational basis
    computational_basis =  [f'{i:06b}' for i in range(2**6)]
    probabilities = np.abs(s)**2
    return np.random.choice(computational_basis, how_many_repetitions, p=probabilities)
```

[8]:
```
# If you're not familiar with what computational basis is,
# it is the basis composed of all 2^n computational basis states :D.
# Each state can be represented by a string of length n composed of '0's or '1's (there's 2^n such strings ;)).
# These are tensor product states. The value at i-th possition tells you whether i-th qubit is in state |0> or |1>.
computational_basis = [f'{i:06b}' for i in range(2**6)]
np.array(computational_basis)
```

[8]:
```
array(['000000', '000001', '000010', '000011', '000100', '000101',
       '000110', '000111', '001000', '001001', '001010', '001011',
       '001100', '001101', '001110', '001111', '010000', '010001',
```

```
'010010', '010011', '010100', '010101', '010110', '010111',
'011000', '011001', '011010', '011011', '011100', '011101',
'011110', '011111', '100000', '100001', '100010', '100011',
'100100', '100101', '100110', '100111', '101000', '101001',
'101010', '101011', '101100', '101101', '101110', '101111',
'110000', '110001', '110010', '110011', '110100', '110101',
'110110', '110111', '111000', '111001', '111010', '111011',
'111100', '111101', '111110', '111111'], dtype='<U6')
```

4 Find the optimal parameters

This is the hardest part of QAOA. Is there an efficient way how to do it? Are you sure your result is precise enough?
Here, let us do it very roughly.

[9]:
```
how_many_repetitions = 100
# let's do grid search
best_mc = 0
for g in np.arange(0,np.pi/2,0.01):
    for b in np.arange(0,np.pi/2,0.01):

        # what's the average mc value with g,b given
        results = list( map(cut, p1_qaoa(n,edges,g,b,how_many_repetitions)) )
        mc = sum(results)/len(results)

        if mc > best_mc:
            optimal_parameters = [g,b]
            best_mc = mc

optimal_parameters
```

[9]:
```
[0.26, 0.45]
```

4.0.1 Now, we can use QAOA to find an approximate solution

[10]:
```
# First, let us apply QAOA 100 times. These are the states we measured.
states = p1_qaoa(n,edges,optimal_parameters[0],optimal_parameters[1],100)
states
```

[10]:
```
array(['000111', '111000', '111100', '110010', '100111', '110110',
       '100110', '000101', '001101', '010101', '111000', '000110',
       '100110', '010000', '110010', '100011', '100110', '000110',
       '100101', '000111', '100011', '110000', '111000', '101100',
       '100110', '111101', '100110', '011001', '000011', '100001',
       '100111', '000011', '100111', '000000', '111100', '010101',
       '001001', '111000', '010001', '110010', '000011', '001011',
       '011101', '100110', '111001', '111000', '000111', '100111',
```

```
'111000', '110100', '011010', '100101', '110100', '001011',
'110001', '001011', '100001', '001101', '010001', '001101',
'011010', '000011', '011001', '000111', '100001', '111110',
'000110', '001010', '101010', '110010', '001101', '111001',
'000101', '001101', '001011', '110000', '011001', '101101',
'111000', '001011', '100110', '110100', '001101', '110110',
'011001', '111000', '110010', '010011', '010011', '011010',
'001010', '100110', '111010', '001111', '001011', '000111',
'111000', '011001', '100101', '000101'], dtype='<U6')
```

[11]:
```
# Calculate the cut value for each measured state. Observe the distribution of␣
↪the values.
cuts = list(map(cut,states))
print(f'cut value: \t how many times')
for c in range(8):
    print(f'{c:>9}: {cuts.count(c):>20}')
```

```
cut value:       how many times
        0:                    1
        1:                    0
        2:                    0
        3:                    2
        4:                   14
        5:                   15
        6:                   41
        7:                   27
```

Of course, when using QAOA to find a good enough solution, we would pick the best one from the obtained. So, we would have found the optimal solution in this case. But we would measure the optimal solution only in about a quarter of the time.

5 Approximation ratio

The ratio between the result obtained by the algorithm and the optimal result.

[12]:
```
average_cut = sum(cuts)/len(cuts)
average_cut
```

[12]:
```
5.72
```

[13]:
```
average_cut/7
```

[13]:
```
0.8171428571428571
```

5.1 Very roughly: we could expect the obtained cut to be at least 80% of the optimal.

To find the optimal solutions to hard problems, one may need to iterate through all the possible solutions, which there are exponentially many. This may very soon become intractable. That is the

reason why there are approximate algorithms (like QAOA). A good enough solution is way better than none.

But let me note that so far, classical counterparts do perform better than QAOA.